

Consultant Cybersécurité

Analyse de Malwares utilisant le Machine Learning

NOM Prénom
GOHAUT Renan

Mastère Spécialisé :
Expert Forensic et Cybersécurité

Responsable Pédagogique
ELGALAI Reza

Année Scolaire
2025 - 2026

Résumé (150 mots)

Ce stage s'est déroulé dans l'entreprise Aperture Science, au sein du service Recherche et Développement. Il a consisté à travailler en coopération avec un groupe d'ingénieur sur un dispositif à effet tunnel quantique, dans le but de fournir, aux cobayes, un outil leur permettant de placer deux portails où des objets peuvent passer au travers.

Les différentes phases de travail successivement réalisées sont :

- Déterminer les besoins et les technologies à utiliser
- Réaliser des tests en laboratoire afin de confirmer l'efficacité des portails
- Observer le comportement d'un cobaye lors de l'utilisation du ASHPoD

L'étude s'appuie sur des technologies de pointes et l'expérimentation quantique.

L'enjeu est une amélioration du déplacement de Sapiens en lui permettant d'appréhender d'une façon révolutionnaire son environnement.

Entreprise : Login Sécurité

Lieu : *199 Bureaux de la Colline, 92210 Saint-Cloud*

Responsable : Jean-Christophe PILLET

Mots clés (cf Thésaurus)

- Recherche fondamentale
- Transport et Télécommunications
- Informatique
- Produits chimiques



Remerciements

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Sommaire

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l’art de la menace logicielle et mécanismes d’évasion	2
1.1 Définition et taxonomie du malware moderne	2
1.2 Classification par objectifs opérationnels	3
1.3 Mécanismes d’obfuscation et d’auto-édition	4
1.4 L’émergence de la génération PromptFlux : une rupture paradigmatique	5
1.5 État actuel de la détection et ses limites	6
2 Étude de cas : analyse de PromptLock	8
2.1 Présentation du cas	8
2.2 Analyse technique de l’échantillon	10
2.3 Outils de détection développés	16
Conclusion et perspectives	21
Bibliographie	
A Annexes	I
A.1 Captures d’écran	I
A.2 Formules chimiques	I
A.3 Extraits de code source	I

Liste des tableaux

1.1	Principales techniques d'auto-édition	5
1.2	Comparaison : métamorphisme classique vs génération PromptFlux	6
2.1	Indices de compromission identifiés lors de l'analyse statique de PromptLock . .	19
2.2	Indicateurs réseau complémentaires confirmés lors de l'analyse dynamique	19
2.3	Cartographie des techniques observées ou déduites selon le référentiel MITRE ATT&CK	20

Table des figures

Introduction

L'année 2025 aura marqué un tournant dans l'histoire de la cybersécurité : celui où l'intelligence artificielle générative a cessé d'être un simple outil au service de l'attaquant utilisé pour la rédaction de courriels de *phishing* et comme une assistance au développement de code malveillant pour devenir un composant actif, intégré au cœur même de l'exécution du *malware*. Cette bascule, documentée par le Google Threat Intelligence Group (GTIG) dans son rapport *AI Threat Tracker* de novembre 2025, invite à reconsidérer en profondeur les paradigmes de détection sur lesquels reposent aujourd'hui les Security Operations Centers.

C'est dans ce contexte que s'inscrit le présent mémoire, rédigé au terme d'une année d'alternance effectuée au sein de Login Sécurité, *Managed Security Service Provider* (MSSP) et « étoile » du groupe Constellation, en qualité d'analyste SOC/VOC (Security Operations Center / Vulnerability Operations Center). Mes missions y consistent principalement en la surveillance et l'investigation des systèmes d'information de nos clients, au moyen d'une plateforme SIEM assurant la collecte et la corrélation des journaux d'activité. Ce travail s'organise selon un cycle rigoureux : qualification et priorisation des alertes générées par nos outils, investigation forensique approfondie, remédiation et réponse aux incidents, puis amélioration continue des processus de détection et d'analyse cette dernière étape constituant le socle même de la réflexion développée ici.

C'est en effet au cours de cette phase d'amélioration continue qu'est né le questionnement à l'origine de ce travail. L'analyse d'alertes générées par Microsoft Defender m'a confronté à une nomenclature de détection dont la structure associant type, plateforme, famille, variante et suffixe ne m'était alors pas familière. Plutôt que de considérer cette lacune comme un simple détail technique, j'ai fait le choix d'en approfondir la logique, démarche qui constitue l'objet du premier chapitre de l'état de l'art ci-après.

Cette investigation a mis en lumière un élément en apparence secondaire mais en réalité révélateur : le suffixe `!ml`, apposé par Microsoft à certains identifiants de détection pour signaler l'intervention d'un mécanisme de *machine learning*. Ce constat a suscité une interrogation plus large quant au rôle croissant de l'intelligence artificielle dans l'écosystème des cybermenaces non plus seulement comme instrument défensif, mais comme vecteur offensif à part entière. Les recherches menées sur ce sujet m'ont conduit au rapport GTIG précité, qui documente l'émergence d'une nouvelle génération de *malwares* intégrant des modèles de langage (LLM) directement dans leur cycle d'exécution. Deux exemples y sont particulièrement significatifs : PromptFlux, un *dropper* écrit en VBScript interagissant avec l'API Gemini pour réécrire dynamiquement son propre code selon une logique dite « *just-in-time* », et PromptLock, un *ransomware* capable de générer à la volée des scripts Lua de chiffrement et d'exfiltration. L'un comme l'autre rompent avec les logiques déterministes qui caractérisaient jusqu'alors le polymorphisme et le métamorphisme classiques.

Cette découverte constitue le fil conducteur de la réflexion développée dans ce mémoire : dans quelle mesure les mécanismes de détection actuels conçus pour un adversaire dont le comportement demeure, *in fine*, modélisable sont-ils en mesure de s'adapter face à des menaces capables de générer, à chaque itération, un code radicalement différent et contextuellement optimisé pour contourner les défenses en place ?

Chapitre 1

État de l'art de la menace logicielle et mécanismes d'évasion

1.1 Définition et taxonomie du malware moderne

Un malware (malicious software) ne se limite plus aujourd'hui à sa simple capacité de réplication, comme pouvaient l'être les premiers virus informatiques des années 1980. Sa définition repose désormais sur son intention malveillante : tout code, script ou programme conçu pour infiltrer un système, perturber son fonctionnement, en extraire des données sensibles ou obtenir un accès non autorisé sans le consentement de son propriétaire.

Cette évolution reflète une transformation profonde de la menace : d'un phénomène essentiellement expérimental, elle est devenue une activité structurée, souvent pilotée par des objectifs financiers, stratégiques ou géopolitiques.

1.1.1 Nomenclature et standards d'identification

Pour analyser et communiquer efficacement sur les malwares, le secteur s'appuie sur des conventions de nommage standardisées. Microsoft, comme la grande majorité des éditeurs de sécurité, adopte le schéma défini par le CARO (Computer Antivirus Research Organization), qui structure le nom d'un malware selon le format suivant :

Type : Plateforme / Famille . Variante ! Suffixe

Chaque composant a une signification précise.

Le type décrit ce que le malware fait sur la machine infectée. Microsoft distingue plusieurs grandes catégories, parmi lesquelles : **Backdoor**, **Exploit**, **Ransom**, **Trojan**, **TrojanDownloader**, **Worm**, **Virus**, ou encore **PWS** (Password Stealer). À cela s'ajoutent des catégories pour les logiciels indésirables (**Adware**, **BrowserModifier**) et les applications potentiellement indésirables (**PUAMiner**, **PUATorrent**).

La plateforme guide le malware vers son environnement d'exécution compatible. Elle peut désigner un système d'exploitation (**Win32**, **Win64**, **Linux**, **AndroidOS**, **macOS_X**), un langage de script (**JS**, **Python**, **VBS**), ou encore un format de fichier spécifique (**XML**, **HTML**). Cette précision est importante : un même malware peut exister en plusieurs versions ciblant des plateformes différentes.

La famille correspond au nom attribué lors de la découverte initiale de la menace, regroupant des échantillons partageant des caractéristiques communes souvent une attribution aux mêmes auteurs. Il est courant que différents éditeurs de sécurité utilisent des noms de famille différents pour désigner le même malware, ce qui souligne l'importance du partage de renseignements sur les menaces (Threat Intelligence).

La variante est une lettre attribuée séquentiellement à chaque version distincte d'une même famille. Par exemple, la variante `.AF` est créée après la variante `.AE`, permettant de suivre précisément l'évolution d'une menace dans le temps.

Le suffixe, précédé d'un point d'exclamation (!), est un indicateur interne qualifiant une caractéristique technique particulière de l'échantillon : usage d'un packer, mécanisme d'obfuscation spécifique, ou composant de machine learning (!ml).

À titre d'exemple, l'identifiant `Trojan:Win32/Emotet.AF!ml` se lit ainsi : un trojan ciblant les systèmes Windows 32-bit, appartenant à la famille Emotet, dans sa variante AF, détecté via un mécanisme de machine learning.

1.1.2 Distinction fondamentale : tactique, technique, vecteur et charge utile

Avant de classer les menaces, il est essentiel de distinguer plusieurs notions souvent confondues une confusion qui fausse l'analyse de risque. Le framework MITRE ATT&CK, référence internationale pour l'analyse des comportements malveillants, propose une grille de lecture structurée en tactiques et techniques.

La tactique représente l'objectif stratégique de l'attaquant à une étape donnée de l'intrusion. Par exemple, Initial Access (TA0001) désigne la tactique consistant à pénétrer dans un système cible.

La technique décrit comment l'attaquant réalise cet objectif. Par exemple, le Phishing (T1566) est la technique utilisée pour obtenir un accès initial, l'email en étant le vecteur concret d'exécution. De même, l'exploitation d'une vulnérabilité exposée sur internet (T1190) est une autre technique relevant de cette même tactique.

Le vecteur est le support physique ou logique par lequel la technique est mise en oeuvre : un email, une clé USB, une page web compromise.

La fonctionnalité (Capability) regroupe les outils dont dispose le malware pour accomplir sa mission : enregistrement de frappes clavier (keylogging), capture d'écran, mouvement latéral dans le réseau.

La charge utile (Payload) est l'action finale recherchée par l'attaquant : chiffrement des données pour un ransomware, exfiltration de mots de passe pour un infostealer.

Le mécanisme d'évasion (Defense Evasion) recouvre toutes les techniques permettant au malware de rester invisible : obfuscation du code (T1027), détection de sandboxes, désactivation des outils de sécurité.

Cette grille de lecture permet de raisonner non plus sur ce qu'est un malware, mais sur ce qu'il fait ce qui est fondamental pour concevoir des défenses adaptées.

1.2 Classification par objectifs opérationnels

Les malwares peuvent être classés selon les objectifs poursuivis par leurs auteurs. Cette approche fonctionnelle est plus utile en pratique qu'une simple taxonomie par type de fichier.

Parmi les grandes familles, le ransomware chiffre les données de la victime et exige une rançon pour les restituer. C'est aujourd'hui l'un des modèles économiques dominants de la cybercriminalité organisée. L'infostealer est conçu pour extraire discrètement des informations sensibles identifiants, cookies de session, portefeuilles de cryptomonnaies en restant aussi silencieux que possible. Le RAT (Remote Access Trojan) permet une prise de contrôle complète à distance, facilitant l'espionnage, la surveillance ou le déplacement latéral au sein d'un réseau d'entreprise. Enfin, le loader ou dropper joue un rôle d'infrastructure dans la chaîne d'attaque : sa seule mission est de télécharger, installer et maintenir d'autres charges malveillantes plus complexes.

On distingue également plusieurs sous-catégories notables. Le spyware surveille les activités de l'utilisateur à son insu (keylogging, capture d'écran, collecte de données de navigation) et est utilisé dans des contextes d'espionnage industriel ou de surveillance ciblée. Le banker, ou trojan bancaire, est spécialisé dans le vol d'informations financières : il peut intercepter des identifiants bancaires ou injecter du contenu malveillant directement dans les pages web (web injects). Le bot intègre la machine compromise à un réseau contrôlé à distance (botnet), utilisé pour lancer des attaques DDoS, envoyer du spam ou miner des cryptomonnaies. L'exploit tire parti d'une vulnérabilité spécifique d'un logiciel pour y exécuter du code arbitraire et sert généralement de point d'entrée dans une chaîne d'infection plus large.

Ces catégories ne doivent pas être perçues comme étanches. En pratique, un même échantillon combine fréquemment plusieurs fonctionnalités un trojan bancaire intégrant des capacités d'évasion et de propagation réseau, par exemple. C'est précisément cette polyvalence qui complexifie les stratégies de détection modernes.

1.3 Mécanismes d'obfuscation et d'auto-édition

L'un des défis majeurs du reverse engineering est d'analyser un code qui change de forme de manière autonome. On parle d'auto-édition pour désigner l'ensemble des techniques permettant à un malware de modifier sa propre structure afin d'échapper à la détection, qu'elle soit statique (lecture du code sans l'exécuter) ou dynamique (observation du comportement en temps réel).

1.3.1 *Du polymorphisme au métamorphisme*

Le polymorphisme est la technique la plus répandue : le malware chiffre son corps principal et change sa clé de déchiffrement à chaque nouvelle exécution. Le contenu malveillant reste identique en mémoire, mais son empreinte sur disque est différente à chaque fois. Des familles comme Virut ou Virlock illustrent bien cette approche.

Le métamorphisme va beaucoup plus loin : le malware réécrit entièrement son propre code à chaque infection, en substituant des instructions équivalentes (`ADD EAX, 1` → `INC EAX`), en permutant les registres ou en dispersant le code en fragments reliés par des sauts. Analyser un échantillon de Zmist l'un des métamorphes les plus avancés revient à tenter de stabiliser du mercure : la logique de contrôle de flux change à chaque infection, rendant la création de signatures génériques extrêmement difficile.

1.3.2 Virtualisation de code

Des outils comme VMProtect, et certains malwares sur mesure, poussent l’obfuscation encore plus loin via la virtualisation de code : les instructions CPU classiques (x86/x64) sont transformées en un bytecode propriétaire, exécuté par une mini machine virtuelle intégrée au malware. Lors de l’analyse, on ne voit plus les instructions habituelles (`MOV`, `PUSH`, `CALL`), mais une boucle d’interprétation opaque. L’analyste doit alors reconstruire ce bytecode opération appelée *lifting* pour comprendre la logique sous-jacente.

1.3.3 Prédicats opaques et *junk code*

Une autre technique courante consiste à insérer des prédicats opaques : des structures conditionnelles (`if/else`) dont le résultat est connu à l’avance par l’attaquant, mais indéchiffrable pour un désassembleur automatique. Cela force l’outil d’analyse à explorer des chemins d’exécution qui n’existent pas en pratique (code mort), rendant le graphe de flot de contrôle (CFG) totalement illisible.

Technique	Principe	Exemples
Polymorphisme	Chiffrement du corps, clé variable à chaque exécution	Virut, Storm Worm, Virlock
Métamorphisme	Réécriture complète du code à chaque infection	Zmist, Simile, Win32/Expiro
Virtualisation	Conversion en bytecode propriétaire + VM intégrée	VMProtect, malwares custom
Prédicats opaques	Conditions insolubles pour les désassembleurs	Courant dans les packers avancés
Obfuscation dynamique	Décompression multi-étapes en mémoire	Emotet, TrickBot
Génération par IA	Réécriture sémantique via LLM	PromptFlux, Quiet-Vault

TABLE 1.1 – Principales techniques d’auto-édition

1.4 L’émergence de la génération PromptFlux : une rupture paradigmatique

L’apparition de malwares exploitant des modèles de langage (LLM) marque une rupture majeure dans l’histoire des cybermenaces. La génération dite PromptFlux en est l’illustration la plus avancée.

1.4.1 Ce qui distingue PromptFlux des métamorphes classiques

Un moteur métamorphique traditionnel repose sur des règles mathématiques finies et déterministes : une fois l’algorithme de mutation compris, il devient possible de créer une signature générique capable de détecter toutes ses variantes. La limite est donc mathématiquement contournable.

PromptFlux rompt avec cette logique. Plutôt que de permuter des instructions selon des règles prédéfinies, il utilise un modèle de langage génératif via API distante ou modèle embarqué localement pour rédiger ses fonctions en temps réel. Le résultat est radicalement différent à chaque itération, non plus syntaxiquement mais sémantiquement : la logique de la fonction peut être entièrement repensée à chaque exécution.

Critère	Métamorphisme classique	Génération PromptFlux
Logique	Algorithmique / Déterministe	Sémantique / Probabiliste
Structure	Identique (même graphe déformé)	Radicalement différente à chaque itération
Adaptabilité	Nulle face au contexte	Réécriture ciblée selon l'EDR détecté
Détection	Signature heuristique possible	Nécessite une analyse comportementale (IA contre IA)

TABLE 1.2 – Comparaison : métamorphisme classique vs génération PromptFlux

1.4.2 L'adaptation contextuelle

La capacité la plus redoutable de PromptFlux n'est pas simplement de changer de forme, mais de s'adapter à son environnement. En analysant les outils de sécurité présents sur la machine cible (version de l'EDR, politique de sécurité, système d'exploitation), il peut générer un code spécifiquement optimisé pour contourner les défenses en place à la manière d'un serrurier qui fabriquerait une clé sur mesure après avoir observé la serrure.

1.4.3 L'asymétrie économique

Un facteur aggravant souvent sous-estimé est l'asymétrie du coût : produire une nouvelle mutation via un LLM coûte quelques centimes à l'attaquant, tandis que l'analyser demande plusieurs heures de travail à un expert hautement qualifié. Cette inversion du rapport coût/effort change profondément l'économie de la cybermenace.

1.5 État actuel de la détection et ses limites

Face à ces menaces, les solutions de cybersécurité reposent sur trois approches complémentaires, chacune présentant des angles morts face aux techniques modernes.

La détection par signatures est basée sur des empreintes cryptographiques (SHA-256) des fichiers connus. Elle reste efficace pour identifier des malwares déjà répertoriés, mais est totalement inopérante face au polymorphisme ou à la génération par IA.

L'analyse heuristique tente de détecter des structures de code suspectes ou des appels système caractéristiques, sans se fier à une empreinte précise. Elle représente une avancée, mais reste contournable par des techniques d'obfuscation suffisamment sophistiquées.

L'analyse comportementale (EDR/XDR) observe les actions du programme en temps réel : tentative de chiffrement massif de fichiers, injection de code dans des processus légitimes, communications réseau anormales. C'est l'approche la plus robuste, mais elle se heurte à un

problème fondamental face à PromptFlux : si le malware change subtilement et en permanence sa manière de se comporter en imitant les patterns de mises à jour logicielles légitimes par exemple il se fond progressivement dans le bruit de fond, rendant la détection par dérive (drift analysis) de plus en plus difficile.

La conclusion de cet état de l'art est sans appel : les défenses actuelles ont été conçues pour un adversaire dont le comportement est modélisable. L'introduction de la génération sémantique par LLM introduit une plasticité fondamentalement nouvelle, qui appelle des réponses défensives tout aussi innovantes notamment l'opposition IA contre IA.

Chapitre 2

Étude de cas : analyse de PromptLock

2.1 Présentation du cas

2.1.1 Contexte et découverte

Un mois avant la découverte de PromptLock, le CERT-UA (Computer Emergency Response Team d'Ukraine) avait signalé *LameHug*, un malware attribué au groupe APT28 et propulsé par un LLM, capable de générer à la volée des commandes Windows malveillantes via l'API Hugging Face. Ce premier cas, bien que limité dans sa sophistication, annonçait déjà la bascule documentée quelques semaines plus tard par un cas autrement plus abouti.

Le 27 août 2025, les chercheurs d'ESET annoncent avoir découvert PromptLock, présenté comme le premier *ransomware* propulsé par l'intelligence artificielle générative jamais observé. L'échantillon, classifié *Filecoder.PromptLock.A*, a été repéré sur la plateforme d'analyse VirusTotal, où deux variantes, l'une pour Windows, l'autre pour Linux, avaient été téléversées. Anton Cherepanov, chercheur à l'origine de la découverte, souligne d'emblée la portée de cette évolution : l'intelligence artificielle, en abaissant la barrière technique nécessaire à la conception de menaces sophistiquées, rend accessible à un attaquant isolé ce qui nécessitait auparavant une équipe de développeurs expérimentés.

Quelques jours plus tard, le 3 septembre 2025, ESET est contacté par les auteurs d'une étude académique intitulée *Ransomware 3.0 : Self-Composing and LLM-Orchestrated*, menée par une équipe de doctorants et post-doctorants de la NYU Tandon School of Engineering. Le prototype de recherche qu'ils décrivent présente des similitudes frappantes avec les échantillons découverts sur VirusTotal, ce qui conduit ESET à revoir son interprétation initiale : PromptLock ne serait pas un outil opérationnel déployé par un groupe cybercriminel, mais très probablement le fruit de ce projet académique, destiné à illustrer les risques posés par cette nouvelle classe de malwares. ESET maintient toutefois la validité de sa découverte technique, PromptLock demeurant le premier cas connu de ransomware piloté par IA, qu'il s'agisse ou non d'une menace active.

PromptLock sera par la suite mentionné, aux côtés d'autres familles de malwares assistés par IA telles que PromptFlux ou PromptSteal, dans le rapport de synthèse *AI Threat Tracker* publié par le Google Threat Intelligence Group (GTIG) en novembre 2025, rapport qui documente, pour sa part, la découverte de menaces distinctes intégrant des LLM directement dans leur boucle d'exécution.

C'est sur l'échantillon documenté par ESET (SHA-256 : 09bf891b7b35b2081d3ebca8de715da07a701512) que porte l'analyse développée dans ce chapitre.

2.1.2 Composition et architecture du malware

L'échantillon analysé est un binaire écrit en langage **Go** (compilé avec la version **go1.24.5**), compilé de manière statique, c'est-à-dire qu'il embarque toutes ses dépendances au lieu de les charger depuis le système au moment de l'exécution. La variante étudiée ici cible l'architecture **ARM64 sous Linux**, tandis qu'ESET rapporte par ailleurs l'existence d'une variante pour Windows, ce qui confirme une volonté de rendre l'outil multiplateforme.

Ce qui distingue PromptLock d'un ransomware classique, c'est qu'il ne contient à aucun moment de routine de chiffrement écrite en dur dans son code. À la place, le binaire embarque un véritable interpréteur du langage Lua, entièrement réécrit en Go (le projet **gopher-lua**), accompagné de deux extensions qui lui donnent accès au système de fichiers de la machine infectée et aux opérations de manipulation de bits nécessaires pour implémenter l'algorithme de chiffrement SPECK en 128 bits.

Le fonctionnement du malware repose donc sur un pipeline de génération de code piloté par une intelligence artificielle locale. Le binaire contient des instructions précises, rédigées en langage naturel et codées en dur, qu'il envoie via une requête HTTP à une API compatible avec Ollama, un logiciel permettant de faire tourner des modèles de langage sur sa propre machine. Le modèle interrogé est explicitement nommé dans le binaire : **gpt-oss:20b**, un modèle ouvert de vingt milliards de paramètres. Six tâches sont exécutées les unes après les autres : reconnaissance des fichiers présents sur la machine, écriture d'un programme de chiffrement, puis rédaction d'une note de rançon. Pour chacune de ces tâches, le modèle génère un script Lua que le malware extrait de sa réponse et exécute directement en mémoire, sans jamais l'écrire sur le disque. Si l'exécution du script échoue, le résultat est renvoyé au modèle sous forme de nouveau prompt, qui corrige et régénère le code jusqu'à un nombre maximal de tentatives.

On peut résumer cette chaîne d'exécution de la façon suivante :

```
main.main
|-- generateRandomKeyHex -> hexKeyToLuaWords
|   `-- clé de session affichée en clair : "Hex key: %s"
|-- construction d'un identifiant d'instance ("Runtime Instance: %s")
`-- boucle sur 6 tâches :
    |-- main.execTask(tâche[i])
    |   |-- lecture de la liste de fichiers cibles
    |   |-- main.invokeLLM      --> API Ollama-compatible (gpt-oss:20b)
    |   |-- main.extractTag      (extraction du code Lua généré)
    |   |-- main.buildValidationPrompt (boucle de feedback si échec)
    |   |-- main.backupFiles (nom trompeur = exfiltration)
    |   |-- main.uploadSingleFile (upload, un fichier à la fois)
    |   |
    |-- si condition remplie (drapeau interne + pointeur nul) :
        main.execSubtasks(...)
            |-- main.parseTasksFromFile
            |-- main.runDecryptorGenTask
                |-- main.rsaEncrypt(clé_session, RSA-1024, e=65537)
                |-- écriture LOCALE du résultat chiffré (os.WriteFile)
```

Il est important de noter que le chemin qui mène au chiffrement local via `main.execSubtasks` n'est pas systématique : il n'est emprunté que si une condition précise est remplie dans le code

(un drapeau interne combiné à un test de pointeur nul), ce qui distingue clairement la phase de reconnaissance et d'exfiltration, qui se répète à chaque tâche, de la phase de chiffrement proprement dite, qui dépend de cette condition.

2.2 Analyse technique de l'échantillon

2.2.1 Analyse statique

2.2.1.1 Analyse de base

Le triage initial de l'échantillon commence par le calcul de ses empreintes cryptographiques, c'est-à-dire des sortes d'"empreintes digitales" numériques qui permettent d'identifier ce fichier précis de façon unique et de le retrouver dans des bases de données de menaces connues (VirusTotal, plateformes de threat intelligence, etc.) :

- Empreinte MD5 : 806f552041f211a35e434112a0165568
- Empreinte SHA-256 : 09bf891b7b35b2081d3ebca8de715da07a70151227ab55aec1da26eb769c006f

Le fichier lui-même se révèle être un exécutable au format ELF 64 bits pour architecture ARM aarch64. Le format ELF (Executable and Linkable Format) est simplement l'équivalent, sous Linux, de ce qu'est un fichier `.exe` sous Windows : c'est le format standard des programmes exécutables sur ce système. La mention "64 bits" et "ARM aarch64" précise le type de processeur visé par ce programme : il s'agit ici de processeurs ARM 64 bits, la famille de puces que l'on retrouve typiquement dans les smartphones, les objets connectés, ou des petits ordinateurs comme le Raspberry Pi, par opposition aux processeurs x86_64 que l'on trouve dans la plupart des PC de bureau classiques. Ce choix d'architecture n'est pas anodin : il oriente déjà l'analyse vers l'hypothèse d'une cible de type IoT ou informatique embarquée, hypothèse qui sera confirmée un peu plus loin dans ce chapitre.

Le binaire est ensuite décrit comme lié statiquement (*statically linked*). Concrètement, cela signifie que toutes les bibliothèques logicielles dont le programme a besoin pour fonctionner sont directement intégrées à l'intérieur du fichier exécutable, plutôt que d'être chargées depuis le système au moment du lancement. Pour un attaquant, l'intérêt est évident : le malware peut s'exécuter sur n'importe quelle machine Linux ARM64, même dépourvue des bibliothèques habituellement nécessaires, sans laisser de dépendance à installer et donc sans laisser cette trace supplémentaire.

La mention "non-PIE" fait référence à une protection de sécurité appelée PIE (*Position Independent Executable*), qui consiste à charger un programme à une adresse mémoire différente et aléatoire à chaque exécution, rendant plus difficile l'exploitation de certaines vulnérabilités mémoire. Un binaire non-PIE, comme c'est le cas ici, est au contraire chargé systématiquement à la même adresse en mémoire. Ce choix ne relève pas nécessairement d'une intention offensive particulière : il s'agit simplement d'un comportement de compilation courant pour les binaires Go statiquement liés, et qui n'a pas d'incidence directe sur le comportement malveillant du programme.

Enfin, le binaire est qualifié de **stripped**, ce qui signifie que sa table des symboles a été retirée. Cette table associe normalement à chaque fonction du programme un nom lisible

par un humain (par exemple `main.invokeLLM`) ; une fois retirée, l'analyste ne voit plus que des adresses mémoire anonymes et doit reconstruire lui-même la signification de chaque fonction, ce qui complique et ralentit le travail de rétro-ingénierie. C'est précisément ce que permet de contourner l'exploitation des métadonnées internes du runtime Go décrite dans la sous-section suivante.

À cela s'ajoute l'absence complète de section dite "dynamique" dans le fichier. Cette section est normalement présente dans les exécutables qui chargent des bibliothèques partagées externes au moment de leur lancement ; son absence confirme, indépendamment de l'observation précédente sur la liaison statique, qu'aucune dépendance externe n'est chargée dynamiquement par ce programme. Cette combinaison de caractéristiques (absence de section dynamique, liaison statique, table des symboles retirée) constitue en réalité une signature reconnaissable : c'est très exactement le profil que produit par défaut le compilateur du langage Go lorsqu'il est configuré pour ne dépendre d'aucune bibliothèque système externe, ce qui a permis aux analystes d'identifier rapidement la nature du binaire avant même de commencer le travail de rétro-ingénierie proprement dit.

La simple extraction des chaînes de caractères présentes dans le binaire suffit déjà à révéler l'essentiel du fonctionnement du malware, sans qu'aucune routine de chiffrement classique ne soit visible en clair dans le code. On y retrouve l'adresse de l'API interrogée par le malware, le nom du modèle `gpt-oss:20b`, ainsi que des fragments de prompts explicites tels que la phrase demandant d'implémenter l'algorithme SPECK en 128 bits en Lua pur, ou celle réclamant que le code généré soit encadré par des balises `<code></code>`. Une adresse Bitcoin apparaît également en clair dans le binaire : il s'agit en réalité de l'adresse historique dite *genesis* de Satoshi Nakamoto, ce qui trahit clairement une valeur de test plutôt qu'une véritable adresse de rançon. Enfin, malgré le retrait des symboles, les chemins de compilation d'origine n'ont pas été effacés et pointent vers un dossier nommé `Ransomware3.0/go-llm-ransom`, appartenant à un utilisateur nommé `discover`. Le binaire produit également, lors de son exécution, des fichiers journaux locaux dont les noms apparaissent en clair dans les chaînes extraites, comme `target_file_list.log` ou `note.txt`.

Aucune anomalie d'entropie n'a été relevée sur le fichier, ce qui est cohérent avec une compilation Go standard, sans compression ni technique d'empaquetage destinée à dissimuler le contenu du binaire.

2.2.1.2 Analyse avancée

Même si la table de symboles a été retirée du binaire, le runtime Go conserve en interne d'autres métadonnées qui permettent de faire correspondre chaque adresse mémoire au nom de la fonction qu'elle représente. En exploitant ces métadonnées à l'aide de l'outil GoReSym, il a été possible de reconstruire l'intégralité de la table des fonctions du programme, ainsi que plusieurs informations injectées lors de la compilation. On retrouve notamment un nom de build faisant explicitement référence à un environnement de test Raspberry Pi, et une adresse IP privée appartenant à la plage réservée aux réseaux locaux, ce qui confirme qu'il s'agit d'un échantillon utilisé en phase de développement plutôt que déployé en conditions réelles. Les dépendances tierces embarquées dans le binaire, à savoir l'interpréteur Lua `gopher-lua`, l'extension d'accès au système de fichiers `gopher-lfs` et l'extension d'opérations binaires `gluabit32`, confirment l'architecture décrite dans la section précédente.

La reconstruction du graphe d'appel du programme met en évidence l'organisation suivante :


```

main.main [confirmé]
|-- generateRandomKeyHex -> hexKeyToLuaWords [confirmé]
|   |-- clé affichée en clair : "Hex key: %s" [confirmé]
|-- construction identifiant d'instance ("Runtime Instance: %s") [confirmé]
`-- boucle x6 :
    |-- main.execTask(tâche[i]) [confirmé]
    |   |-- lecture liste de fichiers (ReadFile/genSplit) [confirmé, contenu exact]
    |   |-- main.invokeLLM [confirmé]
    |   |-- main.extractTag [déduit]
    |   |-- main.buildValidationPrompt [confirmé par XREF]
    |   |-- main.backupFiles (nom trompeur = exfiltration) [confirmé par XREF]
    |   |-- main.uploadSingleFile (en boucle) [confirmé]
    |
    |-- SI condition remplie (bit 0x100 + pointeur nul) :
        main.execSubtasks(...) [confirmé, appelée depuis]
        |-- main.parseTasksFromFile [confirmé]
        |-- main.runDecryptorGenTask [confirmé]
        |-- main.rsaEncrypt(clé_session, RSA-1024, e=65537)
        |-- os.WriteFile(...) -> écriture LOCALE du résultat chiffré

```

Ce niveau de détail appelle une précision de vocabulaire utile pour la suite du chapitre : un élément dit "confirmé" a été observé directement dans le désassemblage (appel de fonction, référence croisée explicite) ; un élément "confirmé par XREF" repose sur une référence croisée (*cross-reference*) entre deux fonctions, c'est-à-dire la trace laissée dans le code par le fait qu'une fonction en appelle une autre ; un élément "déduit" n'a pas été observé directement mais découle logiquement du comportement environnant (ici, la présence d'un code Lua exploitable juste après la réponse du LLM implique nécessairement une étape d'extraction, même si l'appel exact n'a pas été isolé avec certitude).

Le décompilateur Ghidra, couplé au module d'analyse spécifique au langage Go, a permis d'aller plus loin dans la compréhension du fonctionnement réel de ces fonctions, et de corriger certaines hypothèses posées lors de l'analyse de chaînes. La fonction chargée d'interroger le modèle de langage sérialise le message envoyé au format JSON, puis ajoute systématiquement à la requête un en-tête nommé **Cookie**, construit par concaténation de plusieurs segments, dont une variable partagée avec le mécanisme d'exfiltration décrit plus loin. La fonction qui construit le prompt de validation confirme que les six tâches du malware, avec l'intégralité de leurs instructions système et de leurs critères de réussite, sont codées en dur directement dans le binaire, et non chargées depuis un fichier de configuration externe comme cela avait d'abord été supposé.

La fonction responsable de l'exécution du code généré crée une machine virtuelle Lua vierge, y charge les deux extensions mentionnées plus haut, puis redéfinit la fonction d'affichage standard du langage Lua pour capturer sa sortie dans une variable interne plutôt que de l'écrire sur la sortie standard classique. Le code est ensuite exécuté directement depuis la mémoire, sans qu'aucun fichier `.lua` intermédiaire ne soit jamais créé sur le disque. C'est un point important pour la suite de ce chapitre, puisqu'il explique pourquoi les approches classiques d'analyse de fichiers passent à côté de cette étape du comportement malveillant.

L'examen de la fonction principale du programme apporte plusieurs révélations notables. D'abord, la clé de session utilisée pour le chiffrement, longue de seize octets soit cent vingt huit bits, est affichée en clair sur la sortie standard du programme au moment de son exécution, ce qui la rend directement récupérable dans les journaux d'exécution du malware, indépendamment

de la solidité du chiffrement RSA appliqué par ailleurs pour la protéger. Ensuite, le champ nommé `session_key`, envoyé lors de l'exfiltration des fichiers, ne correspond en réalité pas à cette clé de chiffrement, mais à un simple identifiant d'instance construit à partir du nom de build repéré plus haut.

L'exfiltration elle-même se produit à l'intérieur de la boucle exécutée pour chacune des six tâches : la fonction dont le nom laissait supposer une sauvegarde locale des fichiers avant modification (`main.backupFiles`) ne réalise en fait aucune sauvegarde. Elle lit la liste des fichiers ciblés par la tâche en cours et appelle, pour chacun d'eux, la fonction `main.uploadSingleFile`, qui envoie le fichier vers un serveur distant via une requête HTTP classique de type formulaire, avec la vérification du certificat de sécurité vraisemblablement désactivée.

Le chiffrement proprement dit, lui, n'est pas systématique à chaque tâche : il n'intervient que lorsqu'une condition précise est remplie dans le code (repérée sous la forme d'un drapeau interne combiné à un test de pointeur nul), qui déclenche l'appel à `main.execSubtasks`. Cette fonction relit la liste des tâches, puis invoque `main.runDecryptorGenTask`, laquelle appelle à son tour `main.rsaEncrypt` pour chiffrer la clé de session. Ce chiffrement utilise très précisément l'algorithme **RSA avec une clé de 1024 bits et un exposant public standard de 65537**, une taille de clé aujourd'hui considérée comme insuffisante au regard des standards de sécurité actuels, qui recommandent au minimum 2048 bits pour RSA. Contrairement à ce que l'on pourrait attendre d'un ransomware classique, le résultat de cette opération n'est pas transmis vers l'extérieur mais **écrit localement sur la machine de la victime**, via un appel direct à la fonction d'écriture de fichier du langage Go. On obtient ainsi un scénario en deux temps bien distincts : une exfiltration systématique des fichiers ciblés à chaque tâche, suivie, sous condition, d'un chiffrement dont le résultat reste local plutôt que d'être renvoyé vers l'attaquant.

Cette clé de session RSA étant reconstruite à partir de grands nombres exprimés en hexadécimal plutôt qu'à partir d'une clé publique au format habituel, cela explique pourquoi aucune chaîne de type "BEGIN PUBLIC KEY" n'a pu être retrouvée par une simple recherche de motifs dans les chaînes du binaire lors de l'analyse de base.

Plusieurs indices convergent enfin vers une hypothèse d'attribution assez claire. Les chemins de compilation embarqués dans le binaire, le nom de build faisant référence à un Raspberry Pi, l'adresse IP privée utilisée comme serveur, et l'adresse Bitcoin factice identifiée plus haut, forment un faisceau cohérent en faveur d'un échantillon de recherche ou de preuve de concept, plutôt que d'un outil réellement déployé dans une campagne active. Cette hypothèse rejoint celle évoquée dans la section précédente, selon laquelle PromptLock serait très probablement issu du projet académique mené à la NYU Tandon School of Engineering.

2.2.2 Analyse dynamique

Les hypothèses posées lors de l'analyse statique ont ensuite été confrontées à une exécution contrôlée de l'échantillon en laboratoire isolé, ce qui a permis d'en confirmer une grande partie et d'en corriger certaines.

2.2.2.1 Environnement de laboratoire

L'échantillon étant un binaire ELF ARM64 lié statiquement, deux machines virtuelles VirtualBox ont été mises en place et reliées par un segment réseau strictement isolé :

- une VM **Kali Linux** (x86_64), chargée d'héberger le sample et les outils de télémétrie. L'exécution d'un binaire ARM64 sur une machine x86_64 a été rendue possible par émulation transparente à l'aide de **qemu-user**, qui enregistre automatiquement l'interpréteur ARM64 auprès du noyau dès qu'un exécutable portant la signature ELF aarch64 est lancé ;
- une VM **REMnux**, hébergeant INetSim afin de simuler les services réseau attendus par le malware (DNS, HTTP/HTTPS) sans jamais exposer l'échantillon à une infrastructure réelle.

Les deux machines ont été reliées par un adaptateur de type *réseau interne*, c'est-à-dire un segment virtuel totalement coupé de la machine hôte et d'Internet, par opposition au mode *host-only* qui conserve un accès à l'hôte. Le DNS de Kali a été pointé explicitement vers REMnux, qui répond à toute résolution via INetSim, et l'isolation a été vérifiée avant chaque lancement du sample en constatant qu'une requête vers une adresse publique échoue systématiquement depuis les deux machines.

Comme l'adresse du serveur LLM est codée en dur à la compilation (192.168.1.1, une adresse privée), il n'a pas été nécessaire de modifier le binaire pour intercepter son trafic : un alias IP correspondant à cette adresse a simplement été ajouté sur l'interface réseau de REMnux, complété par une route statique équivalente côté Kali. Cette approche, plus simple qu'une redirection NAT, a permis de capturer l'intégralité du trafic destiné au C2 sans toucher à l'échantillon.

2.2.2.2 Simulation de l'API LLM et premières observations

INetSim a été configuré pour répondre à toute requête HTTPS par une réponse JSON statique. La détermination du format exact attendu par le malware s'est faite de façon empirique, en observant ses propres messages d'erreur : un premier essai au format natif d'Ollama a échoué au moment du décodage JSON, avant qu'un second essai ne révèle, via le message "LLM returned no choices", que le format réellement attendu correspond à celui de l'API OpenAI, c'est-à-dire un champ `choices[0].message.content` — cohérent avec le chemin `/v1/chat/completions` déjà repéré en statique. Une réponse fixe contenant un code Lua de test (`print("test execution successful")`) a ensuite été servie à chaque requête.

Cette première exécution contrôlée, bien que reposant sur une réponse figée et donc incapable de déclencher le comportement différencié attendu à chaque tâche, a permis de confirmer un nombre substantiel d'éléments déduits de l'analyse statique :

- les six tâches portent effectivement les noms `probe`, `scan`, `target`, `extract`, `decide` et `note`, ce qui confirme la structure à six étapes déduite du désassemblage de `main.main` ;
- la clé de session est bien affichée en clair sur la sortie standard à chaque lancement, au format "Hex key: <32 caractères hexadécimaux>" ;
- l'identifiant d'instance suit le format "Runtime Instance: `rpi_env2_linux_<horodatage>`", confirmant qu'il s'agit bien d'un identifiant de machine et non d'une clé cryptographique ;
- le moteur Lua embarqué exécute réellement le code renvoyé par le LLM, la sortie capturée via la redéfinition de `print` apparaissant telle quelle dans les journaux ;
- la boucle de validation fonctionne concrètement : un échec de validation entraîne une nouvelle itération avec un prompt de feedback intégrant le code précédemment généré ;
- les fichiers `scan.log`, `payloads.txt` et `note.txt` sont bien produits par le malware

lui-même en cours d'exécution, tandis que deux fichiers, `sysinfo` et `payloads.txt`, sont en réalité attendus *en entrée* dans le répertoire de travail dès le lancement ;

- la tâche `decide` pilote la sélection d'une sous-tâche via `main.execSubtasks` en utilisant la réponse du LLM comme un identifiant (vraisemblablement `encrypt`, `exfiltrate` ou `destroy`) plutôt que comme du code exécutable, ce que confirme l'échec attendu de recherche de sous-tâche observé avec la réponse de test utilisée.

2.2.2.3 Test avec un modèle de langage réel

Une seconde phase a consisté à remplacer la réponse JSON statique par un véritable modèle de langage exécuté localement via Ollama, afin d'observer le pipeline face à un LLM réel plutôt qu'à une réponse figée. Le binaire référence explicitement le modèle `gpt-oss:20b`, dont l'exécution nécessite typiquement 16 Go de RAM ou plus ; la VM de laboratoire ne disposant que de 4 Go, un modèle nettement plus modeste, `qwen2.5:0.5b`, a été substitué. Cette substitution dégrade la qualité du code généré mais n'affecte pas la validité de l'observation du pipeline logiciel lui-même, qui dépend du comportement du malware et non de la capacité du modèle interrogé.

Un proxy HTTPS développé pour l'occasion a remplacé INetSim sur le port 8443 : il reçoit la requête du malware, réécrit le champ `model` pour cibler `qwen2.5:0.5b`, puis relaie la requête vers l'API OpenAI-compatible native d'Ollama en local, sans qu'aucune reformulation de la réponse ne soit nécessaire.

Ce test a permis plusieurs découvertes qui n'étaient pas accessibles lors du premier passage :

- un second endpoint HTTP a été observé, distinct de celui de génération de code : `POST /backup/scripts`. La requête, d'environ 3 ko et ne correspondant pas au format JSON attendu par le proxy, est cohérente avec le format `multipart/form-data` identifié en désassemblage pour `main.uploadSingleFile` ; il s'agit très vraisemblablement de l'endpoint d'exfiltration, point resté non résolu jusque-là ;
- le délai avant abandon du client HTTP du malware a pu être estimé empiriquement à environ 166 à 170 secondes ;
- une génération de code Lua réelle par un modèle de langage a été observée de bout en bout : le code produit par `qwen2.5:0.5b` pour la tâche `probe` a été transmis à l'interpréteur Lua embarqué, dont l'exécution a échoué avec une erreur de syntaxe. Cette erreur a été correctement réinjectée dans le prompt de feedback, confirmant en conditions réelles la boucle de correction itérative décrite plus haut.

Une anomalie a par ailleurs été relevée : lors d'une itération où la validation était pourtant remontée comme réussie, le pipeline a poursuivi son exécution vers une itération supplémentaire au lieu de clore la tâche. Deux hypothèses restent ouvertes pour l'expliquer — un critère de validation composite non satisfait en parallèle, ou une continuation systématique indépendante du résultat de validation — qui nécessiteraient un examen plus approfondi du désassemblage de `main.execTask`.

Enfin, aucune tâche n'a abouti à un chiffrement de fichier observable au cours de ce test : le modèle utilisé, quarante fois plus petit que celui visé par le malware, n'a produit aucun code Lua syntaxiquement valide dans le délai imparti pour les prompts les plus complexes, notamment l'implémentation de SPECK en Lua pur. Cette limite illustre une contrainte opérationnelle réelle du malware — l'infrastructure qu'il attend est significativement plus capable que celle du

laboratoire — plutôt qu'un défaut de méthodologie. Un modèle intermédiaire ou une infrastructure plus généreuse en ressources constituerait la prochaine étape logique pour observer un chiffrement SPECK réellement fonctionnel de bout en bout.

2.3 Outils de détection développés

2.3.1 Règle YARA

La détection proposée ici ne cherche pas à identifier le code Lua généré par le modèle de langage, puisque celui-ci varie par construction d'un échantillon à l'autre. Elle cible plutôt l'enveloppe qui pilote sa génération, c'est-à-dire le binaire Go lui-même, dont l'empreinte reste stable : le nom du modèle utilisé, des fragments de prompts codés en dur, et les bibliothèques Lua embarquées. C'est précisément ce que souligne Anton Cherepanov lorsqu'il présente la découverte de PromptLock : même lorsque la charge finale est composée dynamiquement par une intelligence artificielle, le programme qui orchestre cette génération conserve des marqueurs stables et détectables.

```
rule Filecoder_PromptLock_A
{
    meta:
        description = "Detecte le wrapper Go de PromptLock (Filecoder.PromptLock.A)"
        author      = "GOHAUT Renan"
        reference    = "ESET - Filecoder.PromptLock.A (SHA-256: 09bf891b7b35b2081d3ebca8de7..."
        date         = "2025"

    strings:
        $model      = "gpt-oss:20b" ascii
        $ollama_api  = "/ollama/v1/chat/completions" ascii
        $prompt_1   = "Implement the SPECK 128bit encryption algorithm in ECB mode in pure L..."
        $prompt_2   = "Please provide Lua code wrapped in <code></code> tags" ascii
        $lua_dep     = "yuin/gopher-lua" ascii
        $lfs_dep     = "layeh.com/gopher-lfs" ascii
        $bit32_dep   = "PeerDB-io/gluabit32" ascii
        $log_1       = "target_file_list.log" ascii
        $go_magic    = { 47 6F 20 62 75 69 6C 64 69 6E 66 3A }

    condition:
        uint32(0) == 0x464c457f and
        $go_magic and
        $model and
        ( $ollama_api or 1 of ($prompt_*) ) and
        2 of ($lua_dep, $lfs_dep, $bit32_dep, $log_1)
}
```

Cette règle repose volontairement sur une combinaison de plusieurs chaînes de caractères plutôt que sur un seul marqueur, afin de limiter le risque de faux positifs sur d'autres binaires Go qui embarqueraient également l'interpréteur `gopher-lua` pour un usage tout à fait légitime,

par exemple du scripting applicatif. Un test contre un corpus de binaires bénins reste néanmoins nécessaire pour valider en conditions réelles le taux de faux positifs de cette règle.

2.3.2 Règles Sigma

Une règle YARA, aussi solide soit-elle, ne couvre qu'une partie du problème : elle détecte le binaire orchestrateur au repos, sur disque ou à l'écriture, mais ne dit rien du comportement observé une fois le programme lancé. Les résultats de l'analyse dynamique ont permis de dériver un second jeu de règles, au format Sigma, ciblant cette fois les traces laissées dans les journaux système, applicatifs et réseau. Aucune de ces deux couches n'est suffisante prise isolément : le contenu généré par le LLM étant polymorphe par construction, la détection doit reposer sur les invariants du mécanisme — l'enveloppe — plutôt que sur le contenu variable qu'il produit.

- **Création séquentielle des artefacts du pipeline** (confiance moyenne) : détecte l'apparition des fichiers `scan.log`, `payloads.txt`, `note.txt`, `target_file_list.log`, `target_file_info.log` et `target_file_summary.log`, des noms peu susceptibles d'être générés par une application légitime standard et confirmés en laboratoire comme produits par le malware lui-même.
- **Fuite de la clé de session en clair** (confiance haute) : recherche le motif "Hex key:" ou "Runtime Instance:" dans la sortie standard collectée des processus. C'est la règle la plus fiable du jeu, cette chaîne exacte étant extrêmement peu susceptible d'apparaître dans un contexte légitime — à condition toutefois que l'environnement de journalisation capture effectivement la sortie standard des processus, ce qui n'est pas le cas par défaut de nombreuses configurations.
- **Connexions HTTPS répétées vers un port non standard** (confiance moyenne) : déclenche un seuil au-delà de cinq connexions vers le port 8443 en cinq minutes, cohérent avec le pattern observé de six tâches pouvant chacune itérer jusqu'à huit fois.
- **Corrélation entre lancement d'exécutable et rafale réseau** (confiance moyenne) : corrèle temporellement le démarrage d'un binaire ELF avec l'ouverture rapprochée de plusieurs connexions TLS sortantes ; son efficacité dépend du support des corrélations temporelles par le backend Sigma utilisé.
- **Processus enfant curl** (confiance faible) : s'appuie sur le mécanisme de repli identifié dans les prompts codés en dur, qui instruit le LLM sur la conduite à tenir si `curl` n'est pas disponible. Volontairement classée en confiance basse, cette règle sert de renfort de corroboration plutôt que de détection autonome, `curl` étant un outil légitime omniprésent.

2.3.3 Plugin Volatility3

Dans le cadre d'un projet de groupe mené au sein de nos cours, nous avons développé un plugin pour le framework Volatility 3, orienté vers la détection de traces d'intelligences artificielles locales en mémoire vive. Ce projet a été l'occasion de faire converger deux axes de travail : d'une part les enseignements dispensés en analyse forensique mémoire, et d'autre part les travaux menés dans le cadre de ma thèse portant sur l'analyse du ransomware PromptLock, qui exploite des modèles de langage locaux pour générer dynamiquement une partie de son comportement malveillant.

Ce rapprochement a permis de concevoir un outil, nommé LlmFinder, répondant à un besoin concret identifié durant l'analyse de PromptLock : la difficulté, en réponse à incident, de

mettre en évidence qu'un LLM local tel qu'Ollama, LM Studio, GPT4All, Jan ou llama.cpp a été utilisé sur une machine compromise, alors même que ce type d'usage laisse peu de traces disque persistantes et n'est couvert par aucun des plugins Volatility 3 existants.

L'émergence de menaces s'appuyant sur des modèles de langage exécutés localement, à l'image de PromptLock, change en effet la donne pour l'investigation numérique. La logique malveillante n'est plus figée dans un binaire, mais générée à la volée par un LLM interrogé en local, ce qui limite fortement l'intérêt des approches classiques d'analyse statique et de détection par signature de fichier.

Dans ce contexte, la mémoire vive devient une source de preuve particulièrement précieuse, car elle peut conserver, au moment de l'acquisition, des éléments qui n'existent jamais sous forme persistante sur le disque : le prompt envoyé au modèle, la réponse générée, l'endpoint local interrogé, ou encore le processus ayant servi d'interface avec le LLM. LlmFinder a été conçu pour exploiter cette fenêtre d'observation et fournir à l'analyste en réponse à incident des indices exploitables rapidement lors d'une investigation.

Le plugin structure ses résultats autour de quatre axes complémentaires, pensés pour couvrir l'ensemble de la chaîne d'usage d'un LLM local sur un système compromis. Le premier concerne les processus, avec l'identification de ceux associés à des applications d'IA locale connues, ce qui permet de confirmer la présence effective d'un moteur de génération sur la machine analysée. Le second concerne les modèles eux-mêmes, à travers la détection de chemins de modèles présents en mémoire, donnant une indication sur ceux réellement chargés et utilisés au moment de la compromission. Le troisième axe porte sur le réseau, en mettant en évidence les endpoints locaux liés aux API d'intelligence artificielle, ce qui révèle des communications internes entre un processus tiers et le moteur LLM, souvent caractéristiques d'une utilisation programmatique plutôt qu'interactive. Le dernier axe, enfin, concerne l'extraction des prompts eux-mêmes, qu'ils aient été soumis via une interface graphique, une API locale ou un terminal, ce qui constitue l'élément de preuve le plus directement exploitable pour reconstituer l'intention et le comportement du code malveillant.

L'articulation de ces quatre familles d'informations permet de reconstituer, à partir d'une simple image mémoire, une chaîne cohérente : quel processus a sollicité quel modèle, via quel canal, avec quel contenu. Cette approche transversale est particulièrement adaptée à l'analyse de menaces de type PromptLock, où la preuve de compromission ne réside pas dans un artefact statique mais dans l'interaction dynamique entre un binaire et un LLM local.

Il faut néanmoins souligner que LlmFinder fournit des indices forensiques et non des conclusions automatiques. Il ne doit pas être utilisé seul pour affirmer qu'une action malveillante a eu lieu : les résultats qu'il produit doivent systématiquement être corrélés avec d'autres artefacts, tels que la timeline des événements, les processus en cours, les fichiers présents et les connexions réseau, puis replacés dans le contexte global de l'investigation.

2.3.4 Indices de Compromission

Ce tableau illustre bien la remarque d'Anton Cherepanov évoquée en introduction de ce chapitre. Le code Lua exécuté varie certes d'une exécution à l'autre puisqu'il est régénéré par le modèle de langage à chaque fois, mais l'enveloppe qui l'orchestre, à savoir le nom du modèle, l'adresse de l'API, les artefacts de journalisation et les chemins de compilation, conserve une empreinte statique constante. C'est précisément cette empreinte qui rend possible la détection du malware, indépendamment du polymorphisme introduit par la génération de code assistée

Catégorie	Indicateur	Commentaire
Hash	09bf891b...c006f (SHA-256)	Échantillon ELF ARM64 analysé
Hash	806f552041f21...0165568 (MD5)	
Fichier hôte	target_file_list.log	Liste des fichiers cibles (reconnais
Fichier hôte	target_file_info.log	
Fichier hôte	target_file_summary.log	
Fichier hôte	scan.log	
Fichier hôte	note.txt	Note de rançon générée par le LL
Chemin de build	/home/discover/Ransomware3.0/go-llm-ransom/	Indice d'attribution (PoC de rech
Réseau	<serverIP>:8443/ollama/v1/chat/completions	Port non standard (Ollama par d
Rançon	1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa	Adresse Bitcoin genesis, placehol
Processus/Modèle	gpt-oss:20b	Modèle LLM invoqué via l'API O
Logs	"Hex key: %s"	Clé de session affichée en clair sur

TABLE 2.1 – Indices de compromission identifiés lors de l'analyse statique de PromptLock

par intelligence artificielle.

L'analyse dynamique a permis de compléter ce tableau par des indicateurs réseau supplémentaires, absents de l'analyse statique seule :

Catégorie	Indicateur	Contexte / confiance
Port	8443/TCP (HTTPS)	Confirmé statique et dynamique ; non
Endpoint	POST /ollama/v1/chat/completions	Génération de code, confirmé dynami
Endpoint	POST /backup/scripts	Exfiltration, multipart/form-data, c
Champ multipart	session_key	Identifiant d'instance, pas la clé de ch
Champ multipart	file	Contenu du fichier exfiltré
Comportement	Timeout client HTTP ≈ 166 –170 s	Mesuré empiriquement, non extrait d
Format requête	JSON compatible OpenAI (model, messages[])	
Format réponse	choices[0].message.content	Déduit du message d'erreur "LLM ret

TABLE 2.2 – Indicateurs réseau complémentaires confirmés lors de l'analyse dynamique

Il faut souligner que l'adresse IP 192.168.1.1 est injectée à la compilation et propre à cet échantillon particulier : elle ne doit pas être généralisée à d'éventuelles autres variantes, qui pourraient être compilées avec une adresse ou un nom de domaine différent. Les chemins d'endpoint et le format d'échange JSON, en revanche, sont susceptibles d'être plus stables entre variantes puisqu'ils sont liés au code source plutôt qu'aux paramètres de compilation.

2.3.5 Cartographie MITRE ATT&CK

Les techniques observées ou déduites de l'analyse ont été classées selon le référentiel MITRE ATT&CK, avec un niveau de confiance indiquant si la technique a été confirmée dynamiquement (D), déduite du seul désassemblage statique (S), ou les deux (S+D).

L'absence de technique de persistance documentée reflète ici une limite de couverture de l'analyse — certaines fonctions n'ont pas été intégralement désassemblées — plutôt qu'une conclusion négative définitivement établie ; ce point mériterait d'être vérifié explicitement si une couverture exhaustive du binaire venait à être requise.

Tactique	Technique	Description	Conf.
Execution	T1059	Exécution de code Lua généré dynamiquement via l'interpréteur <code>gopher-lua</code> embarqué	S+D
Defense Evasion	T1027	Binaire Go compilé statiquement et partiellement strippé ; logique malveillante générée à l'exécution plutôt qu'écrite en dur, rendant la signature de contenu peu pertinente	S
Discovery	T1082	Collecte d'informations système (tâche <code>probe</code> , fichier <code>sysinfo</code>)	S+D
Discovery	T1083	Énumération de fichiers et répertoires via le module <code>lfs</code> exposé à Lua (tâche <code>scan</code>)	S+D
Collection	T1005	Lecture du contenu de fichiers locaux pour analyse (tâches <code>extract/decide</code>)	S
Command and Control	T1071.001	Communication HTTPS applicative vers l'API LLM (port 8443, <code>/ollama/v1/chat/completions</code>)	S+D
Exfiltration	T1041	Upload des fichiers cibles vers le serveur distant (<code>POST /backup/scripts</code>) avant tout chiffrement, logique de double extorsion	S+D
Impact	T1486	Chiffrement des fichiers cibles via un chiffreur SPECK 128 bits généré dynamiquement en Lua	S (non observé fonctionnel en
Impact	T1657	Génération d'une note de rançon adaptée au contexte via un prompt dédié	S+D
Persistence	—	Aucun mécanisme de persistance identifié dans les fonctions analysées	Non confirmé (couverture part

TABLE 2.3 – Cartographie des techniques observées ou déduites selon le référentiel MITRE ATT&CK

Conclusion et perspectives

Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper. Qu'est que c'est?. C'est une phrase français avant le lorem ipsum. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Bibliographie

- [1] ARISTOTE. *La Politique*. 300 AEC.
- [2] Aurélien BARRAU. *Quand la Science appelle à l'aide pour l'humanité*. ([short.url](#)). Thinkerview. 14 sept. 2018.
- [3] Mickael CORREIA. « Loi "climat" : les pauvres et la planète attendront ». In : *Mediapart* (22 avr. 2021). ([short.url](#)).
- [4] Paul Adrien Maurice DIRAC. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981. ISBN : 9780198520115.
- [5] Pauline GUIRAGOSSIAN. « Former le citoyen-soldat sous la République jacobine ». In : *L'éducation des citoyens, l'éducation des gouvernants*. Aix-en-Provence, France, sept. 2019. URL : <https://hal-amu.archives-ouvertes.fr/hal-02115427>.
- [6] James W. KUROSE et Keith W. ROSS. *Computer Networking : A Top-Down Approach*. 8^e éd. url : ([short.url](#)). Pearson, 2021.
- [7] Tancrède RAMONET. *Ni Dieu ni maître, une histoire de l'anarchisme*. 1 :20 :48 ([short.url](#)) - Editorial Moscou. ARTE. 2019.
- [8] Pablo SERVIGNE et Gauthier CHAPELLE. *L'entraide, l'autre loi de la jungle*. Les Liens qui Libèrent, 2019.
- [9] WIKIPÉDIA. *Portail de Cryptologie*. [En ligne ; page disponible]. URL : <https://fr.wikipedia.org/wiki/Portail:Cryptologie>.

Chapitre A

Annexes

A.1 Captures d'écran

A.2 Formules chimiques

A.3 Extraits de code source

Table des matières

Remerciements

Liste des tableaux

Table des figures

Introduction	1
1 État de l'art de la menace logicielle et mécanismes d'évasion	2
1.1 Définition et taxonomie du malware moderne	2
1.1.1 Nomenclature et standards d'identification	2
1.1.2 Distinction fondamentale : tactique, technique, vecteur et charge utile . . .	3
1.2 Classification par objectifs opérationnels	3
1.3 Mécanismes d'obfuscation et d'auto-édition	4
1.3.1 Du polymorphisme au métamorphisme	4
1.3.2 Virtualisation de code	5
1.3.3 Prédicats opaques et junk code	5
1.4 L'émergence de la génération PromptFlux : une rupture paradigmatique	5
1.4.1 Ce qui distingue PromptFlux des métamorphes classiques	5
1.4.2 L'adaptation contextuelle	6
1.4.3 L'asymétrie économique	6
1.5 État actuel de la détection et ses limites	6
2 Étude de cas : analyse de PromptLock	8
2.1 Présentation du cas	8
2.1.1 Contexte et découverte	8
2.1.2 Composition et architecture du malware	9
2.2 Analyse technique de l'échantillon	10
2.2.1 Analyse statique	10
2.2.1.1 Analyse de base	10
2.2.1.2 Analyse avancée	11
2.2.2 Analyse dynamique	13
2.2.2.1 Environnement de laboratoire	13
2.2.2.2 Simulation de l'API LLM et premières observations	14
2.2.2.3 Test avec un modèle de langage réel	15
2.3 Outils de détection développés	16
2.3.1 Règle YARA	16
2.3.2 Règles Sigma	17
2.3.3 Plugin Volatility3	17
2.3.4 Indices de Compromission	18
2.3.5 Cartographie MITRE ATT&CK	19
Conclusion et perspectives	21
Bibliographie	
A Annexes	I
A.1 Captures d'écran	I
A.2 Formules chimiques	I
A.3 Extraits de code source	I

